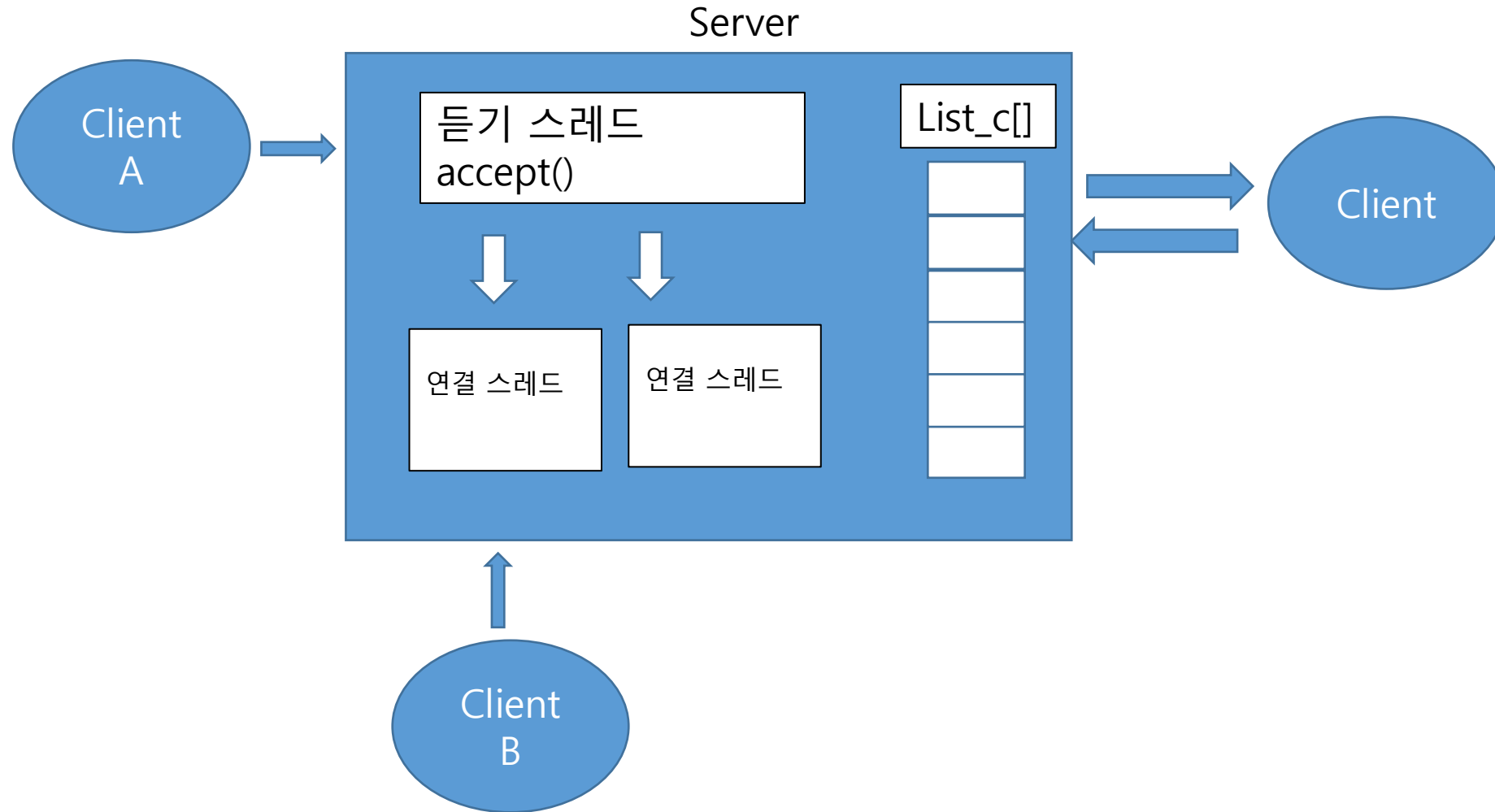


소켓 프로그래밍

13주 채팅 프로그래밍_기능추가(14장)

- 멀티쓰레드 (pthread)

소켓 프로그래밍



소켓 프로그래밍

```
// chat_server_thread.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <pthread.h>

void* do_chat(void *);

pthread_t thread;
pthread_mutex_t mutex;

#define MAX_CLIENT 10
#define CHATDATA 1024

#define INVALID_SOCKET -1

int list_c[MAX_CLIENT];

char escape[] = "exit";
char greeting[] = "Welcome to chatting room\n";
char CODE200[] = "Sorry No More Connection\n";
```

```
main(int argc, char *argv[])
{
    int c_socket, s_socket;
    struct sockaddr_in s_addr, c_addr;
    int len;
    int nfd = 0;
    int i, j, n;
    fd_set read_fds;
    int res;

    if (argc < 2) {
        printf("usage: %s port_number\n", argv[0]);
        exit(-1);
    }

    if (pthread_mutex_init(&mutex, NULL) != 0) {
        printf("Can not create mutex\n");
        return -1;
    }

    s_socket = socket(PF_INET, SOCK_STREAM, 0);

    memset(&s_addr, 0, sizeof(s_addr));
    s_addr.sin_addr.s_addr = htonl(INADDR_ANY);
    s_addr.sin_family = AF_INET;
    s_addr.sin_port = htons(atoi(argv[1]));
```

소켓 프로그래밍

```
if (bind(s_socket, (struct sockaddr *)&s_addr, sizeof(s_addr)) == -1) {
    printf("Can not Bind\n");
    return -1;
}

if(listen(s_socket, MAX_CLIENT) == -1) {
    printf("listen Fail\n");
    return -1;
}

for (i = 0; i < MAX_CLIENT; i++)
    list_c[i] = INVALID_SOCKET;

while(1) {
    len = sizeof(c_addr);
    c_socket = accept(s_socket, (struct sockaddr *)&c_addr, &len);

    res = pushClient(c_socket);
    if (res < 0) {
        write(c_socket, CODE200, strlen(CODE200));
        close(c_socket);
    } else {
        write(c_socket, greeting, strlen(greeting));
        pthread_create(&thread, NULL, do_chat, (void *) c_socket);
    }
}
}
```

```
void *
do_chat(void *arg)
{
    int c_socket = (int) arg;
    char chatData[CHATDATA];
    int i, n;

    while(1) {
        memset(chatData, 0, sizeof(chatData));
        if ((n = read(c_socket, chatData, sizeof(chatData))) > 0) {
            for (i = 0; i < MAX_CLIENT; i++) {
                if (list_c[i] != INVALID_SOCKET) {
                    write(list_c[i], chatData, n);
                }
            }

            if(strstr(chatData, escape) != NULL) {
                popClient(c_socket);
                break;
            }
        }
    }
}
```

소켓 프로그래밍

```
void *
do_chat(void *arg)
{
    int c_socket = (int) arg;
    char chatData[CHATDATA];
    int i, n;

    while(1) {
        memset(chatData, 0, sizeof(chatData));
        if ((n = read(c_socket, chatData, sizeof(chatData))) > 0 ) {
            for (i = 0; i < MAX_CLIENT; i++) {
                if (list_c[i] != INVALID_SOCKET) {
                    write(list_c[i], chatData, n);
                }
            }

            if(strstr(chatData, escape) != NULL) {
                popClient(c_socket);
                break;
            }
        }
    }
}
```

소켓 프로그래밍

```
int
pushClient(int c_socket) {
    int    i;

    for (i = 0; i < MAX_CLIENT; i++) {
        pthread_mutex_lock(&mutex);
        if(list_c[i] == INVALID_SOCKET) {
            list_c[i] = c_socket;
            pthread_mutex_unlock(&mutex);
            return i;
        }
        pthread_mutex_unlock(&mutex);
    }

    if (i == MAX_CLIENT)
        return -1;
}
```

```
int
popClient(int s)
{
    int    i;

    close(s);

    for (i = 0; i < MAX_CLIENT; i++) {
        pthread_mutex_lock(&mutex);
        if ( s == list_c[i] ) {
            list_c[i] = INVALID_SOCKET;
            pthread_mutex_unlock(&mutex);
            break;
        }
        pthread_mutex_unlock(&mutex);
    }

    return 0;
}
```

소켓 프로그래밍

- 서버(chat_server_thread.c) 변경사항

1. list_c 배열의 구조를 변경한다.

- 접속한 클라이언트의 닉네임과 소켓번호를 저장하도록 변경한다.

```
struct user_info
{
    int usersocket;
    char username[MAX_NAME];
};

struct user_info user_list[MAX_CLIENT];
```

소켓 프로그래밍

- 서버(chat_server_thread.c) 변경사항

2. push_Client함수의 매개변수를 변경한다.

Int pushClient(int c_socket) ➔ int pushClient(int c_socket, char *nickname)

3. pop_Client 함수는 매개변수 변경없이 일부코드를 수정한다.

소켓 프로그래밍

```
Int pushClient(int c_socket) {
    int i;

    for (i = 0; i < MAX_CLIENT; i++) {
        pthread_mutex_lock(&mutex);

        //if(list_c[i] == INVALID_SOCKET) {
        //    list_c[i] = c_socket;
        // 소켓번호를 입력하고 클라이언트 닉네임을 username으로 입력한다.

        pthread_mutex_unlock(&mutex);
        return i;
    }
    pthread_mutex_unlock(&mutex);
}

if (i == MAX_CLIENT)
    return -1;
}
```

소켓 프로그래밍

```
Int popClient(int s)
{
    int    i;

    for (i = 0; i < MAX_CLIENT; i++) {
        pthread_mutex_lock(&mutex);
        // if ( s == users[i].usersocket ) {
        //          users[i].usersocket = INVALID_SOCKET;
        // users 배열을 새로운 배열을 사용한다.

        pthread_mutex_unlock(&mutex);
        break;
    }
    pthread_mutex_unlock(&mutex);
}

return 0;
}
```

소켓 프로그래밍

- 서버(chat_server_thread.c) 변경사항
4. pushClient 함수를 사용하기 전에 클라이언트의 소켓번호와 닉네임을 구한다.
- 소켓번호는 기존의 연결소켓(c_socket) 을 사용한다.
 - 닉네임은 클라이언트가 전송(write함수)해준 값을 읽어서(read함수) 사용한다.
- chat_client_thread.c 파일에 다음 코드를 connect 함수 이후에 추가한다.
 write(c_socket, nickname, strlen(nickname));
 // send the nickname to server
 - chat_server_thread.c 파일에 다음을 추가한다.
 n = read(c_socket, nickname, sizeof(nickname));
 nickname[n] = '\0';

소켓 프로그래밍

13주 과제 : 채팅서버 기능추가하기(chat server thread.c)

1. list_c 배열의 구조를 변경한다. user_list 배열로 변경한다.
 2. Push_Client 함수와 pop_Client 함수를 변경한다.
 - 추가될때와 삭제될때 추가/삭제되는 유저이름을 출력한다.
 3. 한명의 클라이언트에게만 메시지를 전송할 수 있도록 변경한다.
 - /user1/ Hello : user1 에게만 전송
 - Hello everyone : 전체 전송
- (추가 4. 클라이언트에서 who 라고 전송하면 로그인된 모든 유저이름을 알려준다..)